how to start a websocket server in dart/flutter

Here are several ways to create a WebSocket server in Dart/Flutter:

1. Using `shelf_web_socket` (Recommended for HTTP Servers)

This is the most common approach when you already have an HTTP server.

Installation

```yaml
dependencies:
  shelf: ^1.4.0
  shelf_web_socket: ^1.0.0
```

Basic Server Code

```dart
import 'dart:io';
import 'package:shelf/shelf.dart';
import 'package:shelf/shelf_io.dart' as io;
import 'package:shelf_web_socket/shelf_web_socket.dart';

void main() async {
  // Create WebSocket handler
  var webSocketHandler = webSocketHandler((webSocket) {
    webSocket.stream.listen(
      (message) {
        print('Received: $message');
        // Echo the message back
        webSocket.sink.add('Echo: $message');

        // Broadcast to all connected clients
        // (you'll need to manage connections in a list)
      },
```

```
      onDone: () => print('Client disconnected'),
    );

    // Send welcome message
    webSocket.sink.add('Welcome to WebSocket Server!');
  });

  // Create shelf pipeline
  var handler = const Pipeline().addHandler(webSocketHandler);

  // Start server
  var server = await io.serve(handler, 'localhost', 8080);
  print('WebSocket server running on ws://localhost:8080');
}
```

2. Using Raw WebSocket from `dart:io`

For more control without external dependencies:

```dart
import 'dart:io';
import 'dart:convert';

void main() async {
  final server = await HttpServer.bind('localhost', 8080);
  print('WebSocket server listening on ws://localhost:8080');

  await for (HttpRequest request in server) {
    if (WebSocketTransformer.isUpgradeRequest(request)) {
      // Handle WebSocket connection
      final webSocket = await WebSocketTransformer.upgrade(request);
      _handleConnection(webSocket);
    } else {
      request.response.statusCode = HttpStatus.forbidden;
      request.response.close();
    }
  }
}

void _handleConnection(WebSocket webSocket) {
  print('Client connected: ${webSocket.hashCode}');
```

```
  webSocket.listen(
    (message) {
      print('Received: $message');

      // Parse JSON if needed
      try {
        final data = jsonDecode(message);
        print('Parsed data: $data');
      } catch (e) {
        // Not JSON, handle as string
      }

      // Send response
      webSocket.add('Server received: $message');

      // Broadcast example
      // broadcastMessage('User said: $message');
    },
    onDone: () {
      print('Client disconnected: ${webSocket.hashCode}');
    },
    onError: (error) {
      print('Error: $error');
    },
  );

  // Send initial message
  webSocket.add('Connected to WebSocket server!');
}
```

3. Managing Multiple Clients

Here's an example with connection management:

```dart
import 'dart:io';
import 'dart:convert';

class WebSocketServer {
  final List<WebSocket> _clients = [];
  final int port;
```

```
WebSocketServer(this.port);

Future<void> start() async {
  final server = await HttpServer.bind(InternetAddress.anyIPv4, port);
  print('WebSocket server running on port $port');

  await for (final request in server) {
    if (WebSocketTransformer.isUpgradeRequest(request)) {
      final webSocket = await WebSocketTransformer.upgrade(request);
      _addClient(webSocket);
    }
  }
}

void _addClient(WebSocket webSocket) {
  _clients.add(webSocket);
  print('New client connected. Total clients: ${_clients.length}');

  webSocket.listen(
    (message) => _handleMessage(webSocket, message),
    onDone: () => _removeClient(webSocket),
    onError: (error) {
      print('WebSocket error: $error');
      _removeClient(webSocket);
    },
  );

  // Send welcome
  webSocket.add(jsonEncode({
    'type': 'connection',
    'message': 'Welcome!',
    'clients': _clients.length
  }));

  // Notify others
  _broadcast(jsonEncode({
    'type': 'join',
    'clients': _clients.length
  }), exclude: webSocket);
}

void _removeClient(WebSocket webSocket) {
  _clients.remove(webSocket);
  print('Client disconnected. Total clients: ${_clients.length}');
```

```
    // Notify remaining clients
    _broadcast(jsonEncode({
      'type': 'leave',
      'clients': _clients.length
    }));
  }

  void _handleMessage(WebSocket sender, dynamic message) {
    print('Message from ${sender.hashCode}: $message');

    // Broadcast to all clients
    _broadcast(jsonEncode({
      'type': 'message',
      'from': sender.hashCode,
      'message': message,
      'timestamp': DateTime.now().toIso8601String()
    }));
  }

  void _broadcast(String message, {WebSocket? exclude}) {
    for (final client in _clients) {
      if (client != exclude && !client.closeCode.isDefinedAndNotNull) {
        client.add(message);
      }
    }
  }
}

void main() async {
  final server = WebSocketServer(8080);
  await server.start();
}
```

4. For Flutter Apps (Client-Side Connection)

To connect to your WebSocket server from Flutter:

```dart
import 'package:flutter/material.dart';
import 'package:web_socket_channel/web_socket_channel.dart';
import 'package:web_socket_channel/io.dart';
```

```
class WebSocketClient extends StatefulWidget {
  @override
  _WebSocketClientState createState() => _WebSocketClientState();
}

class _WebSocketClientState extends State<WebSocketClient> {
  late WebSocketChannel channel;
  final TextEditingController _controller = TextEditingController();
  final List<String> _messages = [];

  @override
  void initState() {
    super.initState();
    _connect();
  }

  void _connect() {
    channel = IOWebSocketChannel.connect('ws://localhost:8080');

    channel.stream.listen(
      (message) {
        setState(() {
          _messages.add('Server: $message');
        });
      },
      onError: (error) => print('Error: $error'),
      onDone: () => print('Connection closed'),
    );
  }

  void _sendMessage() {
    if (_controller.text.isNotEmpty) {
      channel.sink.add(_controller.text);
      _messages.add('You: ${_controller.text}');
      _controller.clear();
      setState(() {});
    }
  }

  @override
  void dispose() {
    channel.sink.close();
    super.dispose();
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('WebSocket Client')),
      body: Column(
        children: [
          Expanded(
            child: ListView.builder(
              itemCount: _messages.length,
              itemBuilder: (context, index) {
                return ListTile(title: Text(_messages[index]));
              },
            ),
          ),
          Padding(
            padding: const EdgeInsets.all(8.0),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _controller,
                    decoration: InputDecoration(hintText: 'Enter message'),
                  ),
                ),
                IconButton(
                  icon: Icon(Icons.send),
                  onPressed: _sendMessage,
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

5. Testing Your Server

You can test using:

1. **Command line with** `wscat`: `npm install -g wscat` then
   `wscat -c ws://localhost:8080`
2. **Browser DevTools**: Open Console and run:

   ```javascript
   const ws = new WebSocket('ws://localhost:8080');
   ws.onmessage = (e) => console.log(e.data);
   ws.send('Hello from browser');
   ```

3. **Postman**: Supports WebSocket connections

Important Considerations

1. **Platform Limitations**:
   - Flutter mobile apps **cannot host** WebSocket servers (iOS/Android restrictions)
   - Use Dart CLI for servers, Flutter only as client
2. **Production Ready**:
   - Add authentication
   - Implement reconnection logic
   - Add message validation
   - Use SSL/TLS (wss://) for production
   - Implement rate limiting
3. **For Production**, consider:
   - `aqueduct` or `conduit` frameworks
   - Redis for pub/sub across multiple servers
   - Load balancer with WebSocket support

Would you like me to elaborate on any specific aspect, such as authentication, scaling, or implementing a specific protocol like Socket.IO?

webSocket.sink.add('Echo: $message');

// Broadcast to all connected clients

```
  // (you'll need to manage connections in a list)
},
onDone: () => print('Client disconnected'),
```

what does on done do here?

Great question! Let me explain what `onDone` does in this context:

`onDone` Callback

`onDone` is called when the WebSocket connection **closes normally** (gracefully). Here's what happens:

When `onDone` triggers:

1. **Client disconnects voluntarily** - Client calls `websocket.close()`
2. **Connection closes cleanly** - Normal WebSocket closure (close code 1000)
3. **Stream completes** - The WebSocket stream reaches its end

Example flow:

```dart
webSocket.stream.listen(
  (message) {
    // Handle incoming messages
    print('Received: $message');
    webSocket.sink.add('Echo: $message');
  },
  onDone: () {
    // This runs when connection closes normally
    print('Client disconnected');

    // You should clean up resources here
    // Example: Remove from active connections list
    connectedClients.remove(webSocket);
    print('Active clients: ${connectedClients.length}');
  },
);
```

Difference between `onDone` and `onError`

```dart
webSocket.stream.listen(
  (message) => handleMessage(message),
  onDone: () {
    // Called for NORMAL closure
    print('Connection closed gracefully');
    cleanupConnection(webSocket);
  },
  onError: (error) {
    // Called for ABNORMAL closure/errors
    print('Connection error: $error');
    emergencyCleanup(webSocket);
  },
  cancelOnError: false, // Whether to stop listening on error
);
```

Complete Example with Connection Management

```dart
import 'dart:io';
import 'package:shelf/shelf.dart';
import 'package:shelf/shelf_io.dart' as io;
import 'package:shelf_web_socket/shelf_web_socket.dart';

// Store active connections
final List<WebSocket> activeConnections = [];

void main() async {
  var webSocketHandler = webSocketHandler((webSocket) {
    print('New client connected: ${webSocket.hashCode}');

    // Add to active connections
    activeConnections.add(webSocket);
    print('Total connections: ${activeConnections.length}');

    // Listen for messages
    webSocket.stream.listen(
      (message) {
```

```
        print('${webSocket.hashCode} says: $message');

        // Echo back to sender
        webSocket.sink.add('You said: $message');

        // Broadcast to ALL other clients
        _broadcastToOthers(
          'User ${webSocket.hashCode}: $message',
          sender: webSocket
        );
      },
      onDone: () {
        // This runs when client disconnects NORMALLY
        print('Client ${webSocket.hashCode} disconnected gracefully');
        _removeConnection(webSocket);
      },
      onError: (error) {
        // This runs on connection errors
        print('Client ${webSocket.hashCode} error: $error');
        _removeConnection(webSocket);
      },
    );

    // Send welcome message
    webSocket.sink.add('Welcome! Your ID: ${webSocket.hashCode}');

    // Notify others about new user
    _broadcastToOthers(
      'New user joined: ${webSocket.hashCode}',
      sender: webSocket
    );
  });

  var handler = const Pipeline().addHandler(webSocketHandler);
  var server = await io.serve(handler, 'localhost', 8080);
  print('Server running on ws://localhost:8080');
}

void _broadcastToOthers(String message, {required WebSocket sender}) {
  for (var client in activeConnections) {
    if (client != sender) {
      try {
        client.sink.add(message);
      } catch (e) {
```

```
        // Remove if we can't send (connection might be dead)
        print('Failed to send to ${client.hashCode}: $e');
        _removeConnection(client);
      }
    }
  }
}

void _removeConnection(WebSocket webSocket) {
  activeConnections.remove(webSocket);
  print('Removed ${webSocket.hashCode}. Remaining: ${activeConnections.length}');

  // Notify remaining users
  _broadcastToOthers('User ${webSocket.hashCode} left');
}
```

Real-World Scenarios

1. Web Page Closes

```javascript
// Client-side JavaScript
const ws = new WebSocket('ws://localhost:8080');
// When user closes browser tab, onDone fires on server
```

2. Mobile App Goes to Background

```dart
// Flutter client
@override
void dispose() {
  channel.sink.close(); // This triggers onDone on server
  super.dispose();
}
```

3. Manual Disconnection

```javascript
// Client explicitly closes
ws.close(1000, "Goodbye"); // Normal closure
```

Important Notes

1. `onDone` **doesn't guarantee immediate execution** - There might be a delay
2. **Check connection state** - Sometimes you should verify:

```dart
onDone: () {
  if (webSocket.closeCode == 1000) {
    print('Normal closure');
  } else {
    print('Closed with code: ${webSocket.closeCode}');
  }
}
```

3. **Resource cleanup** - Always clean up in both `onDone` and `onError`:

```dart
void cleanupConnection(WebSocket webSocket) {
  // Remove from connections list
  // Cancel any pending operations
  // Close any related resources
  // Update UI/notify other users
}
```

Common Close Codes

- **1000**: Normal closure
- **1001**: Going away (browser tab closed)
- **1006**: Abnormal closure
- **1008**: Policy violation

Bottom line: Use `onDone` for graceful disconnection handling and cleanup. It's your opportunity to properly close resources and update the system state when a client leaves normally.

is it necessary to run the server

Yes, you must run the WebSocket server separately from your Flutter app. Here's why and how:

Why You Need a Separate Server

1. Platform Limitations:

- **Flutter mobile apps CANNOT host servers** on iOS/Android due to:
  - App sandboxing/security restrictions
  - Network permission limitations
  - Background execution constraints
- **Flutter web apps CANNOT host WebSocket servers** in browsers
- Only **Flutter desktop apps** (Windows/Mac/Linux) might host servers with special permissions

2. Architecture Requirements:

```text
┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐
│   Flutter App   │────►│  WebSocket      │────►│  Flutter App    │
│   (Client #1)   │     │  Server         │     │  (Client #2)    │
│   Mobile/Web    │◄────│  (Separate      │◄────│  Mobile/Web     │
└─────────────────┘     │   Process)      │
```